SIMATS Engineering • CSE • CSA0804 — Python Programming • 14-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

---

Student 3 **Alidena Tharun Kumar Reddy** 192572068RD 10 / 10 submitted

Q1. Selection Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

nums = list(map(int, input().split()))
nums = selection_sort(nums)
print(nums)
```

INPUT `23 78 45 8 32 56`

OUTPUT `[8, 23, 32, 45, 56, 78]`

Q2. Insertion Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
arr = list(map(int, input().split()))

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1
    while j ≥ 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1
    arr[j + 1] = key

print(arr)
```

INPUT `8 5 7 1 9 3`

OUTPUT `[1, 3, 5, 7, 8, 9]`

SIMATS Engineering • CSE • CSA0804 — Python Programming • 14-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q3. Merge Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
def merge_sort(arr):
    if len(arr) ≤ 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    i = j = 0
    merged = []

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    while i < len(left):
        merged.append(left[i])
        i += 1

    while j < len(right):
        merged.append(right[j])
        j += 1

    return merged

arr = list(map(int, input().split()))
sorted_arr = merge_sort(arr)
print(sorted_arr)
```

INPUT

```
33 27 43 3 9 82 10
```

OUTPUT

```
[3, 9, 10, 27, 33, 43, 82]
```

```
def is_diagonal_matrix(matrix):
    rows = len(matrix)
    cols = len(matrix[0])

    # A diagonal matrix must be square
    if rows ≠ cols:
        return False

    for i in range(rows):
        for j in range(cols):
            # If we are NOT on the main diagonal
            if i ≠ j:
                # Check if the element is non-zero
                if matrix[i][j] ≠ 0:
                    return False
    return True

# Taking input
# Since the sample shows list format, we use eval to parse the string into a list
try:
    matrix_input = eval(input())
    if is_diagonal_matrix(matrix_input):
        print("Diagonal Matrix")
    else:
        print("Not Diagonal Matrix")
except EOFError:
    pass
```

INPUT

```
1 0 0
0 5 0
0 0 8
```

OUTPUT

```
Security Error: Disallowed code pattern detected.
```

Q5. Row and Column Sums

CODE

```
matrix = eval(input())

# Calculate row sums using a list comprehension
row_sums = [sum(row) for row in matrix]

# Calculate column sums using a list comprehension
col_sums = [sum(matrix[row][col] for row in range(len(matrix))) for col in range(len(matrix[0]))]

print(f"Row= {row_sums}")
print(f"Col= {col_sums}")
```

INPUT

```
1 2
3 4
```

OUTPUT

```
(no output)
```

SIMATS Engineering • CSE • CSA0804 — Python Programming • 14-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q6. Maximum Subarray Sum

Score: 6.67 **TC: 2/3 passed**

CODE

```
import re

def max_subarray_sum(arr):
    max_so_far = arr[0]
    current = arr[0]

    for x in arr[1:]:
        current = max(x, current + x)
        max_so_far = max(max_so_far, current)

    return max_so_far

def main():
    s = input().strip()

    # Extract integers even if input has no spaces like: "1-3 4 -1"
    nums = list(map(int, re.findall(r'-?\d+', s)))

    print(max_subarray_sum(nums))

if __name__ == "__main__":
    main()
```

INPUT
```
-2 1 -3 4 -1 2 1-5 4
```

OUTPUT
```
6
```

Q7. Common Elements Among Three Lists

Score: 10.00 **TC: 3/3 passed**

CODE

```
def common_elements_three_lists(inp: str):
    parts = inp.strip().split(";")
    a = list(map(int, parts[0].split())) if parts[0].strip() else []
    b = list(map(int, parts[1].split())) if parts[1].strip() else []
    c = list(map(int, parts[2].split())) if parts[2].strip() else []

    common = sorted(list(set(a) & set(b) & set(c)))
    return common

# Read input exactly as in the problem (three lists separated by ';')
s = input()
ans = common_elements_three_lists(s)
print(ans)
```

INPUT
```
1 2 3;2 3 4;2 5
```

OUTPUT
```
[2]
```

SIMATS Engineering • CSE • CSA0804 — Python Programming • 14-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q8. Matrix Multiplication

CODE

```
import ast

def matmul(A, B):
    # A: m x n, B: n x p
    m, n = len(A), len(A[0])
    n2, p = len(B), len(B[0])

    if n ≠ n2:
        raise ValueError("Matrix multiplication not possible: A columns must equal B rows.")

    # Result: m x p
    result = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            s = 0
            for k in range(n):
                s += A[i][k] * B[k][j]
            result[i][j] = s
    return result

# Read entire input and parse A and B
s = input().strip()

# Support input formats like:
# A=[[1,2],[3,4]] B=[[5,6],[7,8]]
# or with spaces/newlines.
parts = s.split("B=")
A_part = parts[0].strip()
B_part = parts[1].strip() if len(parts) > 1 else ""

A_str = A_part.replace("A=", "").strip()
B_str = B_part.strip()

A = ast.literal_eval(A_str)
B = ast.literal_eval(B_str)

C = matmul(A, B)

# Print in the expected nested list format
for row in C:
    print(row)
```

OUTPUT

```
(no output)
```

SIMATS Engineering • CSE • CSA0804 — Python Programming • 14-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q9. Sort Rotate Min Max

Score: 3.33 **TC: 1/3 passed**

CODE

```
import sys

def solve():
    # Read the entire input line
    line = sys.stdin.read().strip()
    if not line:
        return

    # The input format is "List=5 2 8 1 4,k=2"
    # We split by the comma first to separate the list part and the k part
    try:
        parts = line.split(',')

        # Process the list part: "List=5 2 8 1 4"
        list_part = parts[0].split('=')[1]
        # Convert the space-separated string into a list of integers
        lst = [int(x) for x in list_part.split()]

        # Process the k part: "k=2"
        k_part = parts[1].split('=')[1]
        k = int(k_part)

        # 1. Sort the list
        sorted_list = sorted(lst)

        # 2. Rotate the list (Left Rotation based on Sample 1)
        # Sample 1: [1, 2, 4, 5, 8], k=2 → [4, 5, 8, 1, 2]
        # This means we take elements from index k to the end, then add elements from start to k
        n = len(sorted_list)
        if n > 0:
            shift = k % n
            rotated_list = sorted_list[shift:] + sorted_list[:shift]
        else:
            rotated_list = []

        # 3. Find Min and Max
        if n > 0:
            min_val = min(sorted_list)
            max_val = max(sorted_list)
        else:
            min_val = None
            max_val = None

        # Output formatting to match Sample Output exactly
        print(f"Sorted= {sorted_list}")
        print(f"Rotated= {rotated_list}")
        print(f"Min= {min_val}")
        print(f"Max= {max_val}")

    except Exception:
        # In case of unexpected input format, fail silently or handle as needed
        pass

if __name__ == "__main__":
    solve()
```

INPUT

```
list = 5 2 8 1,k=2
```

OUTPUT

```
Sorted= [1, 2, 5, 8]
Rotated= [5, 8, 1, 2]
Min= 1
Max= 8
```

SIMATS Engineering • CSE • CSA0804 — Python Programming • 14-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q10. Sort Matrix Search

CODE

```
def solve():
    try:
        # Reading input for the matrix
        # Format expected: Matrix=[[9,3],[7,1]]
        matrix_input = input().strip()
        find_input = input().strip()

        # Parsing the matrix
        # Extracting content between [[ and ]]
        matrix_str = matrix_input.split('=')[1]
        import ast
        matrix = ast.literal_eval(matrix_str)

        # Parsing the find value
        find_val = ast.literal_eval(find_input.split('=')[1])

        rows = len(matrix)
        cols = len(matrix[0])

        # Step 1: Flatten the matrix
        flat_list = []
        for r in matrix:
            for val in r:
                flat_list.append(val)

        # Step 2: Sort the flattened list
        flat_list.sort()

        # Step 3: Reconstruct the sorted matrix
        sorted_matrix = []
        for i in range(0, len(flat_list), cols):
            sorted_matrix.append(flat_list[i:i+cols])

        # Step 4: Find the position of the number
        position = None
        for r in range(rows):
            for c in range(cols):
                if sorted_matrix
"[c] == find_val:
                    position = (r, c)
                    break
            if position:
                break

        # Output the results
        if position:
            print(f"Sorted= {sorted_matrix}")
            print(f"Position= {position}")
        else:
            print("Not Found")

    except Exception:
        # Fallback for potential parsing errors in different environments
        pass

if __name__ == "__main__":
    solve()
```

INPUT

```
matrix = [[9,3],[7,1], find=7
```

OUTPUT

```
(no output)
```